Hands On Lab Building and Training an HMM for Metamorphic Malware Detection

November 3, 2026

0.1 Hands-On Lab: Building and Training an HMM for Metamorphic Malware Detection

Estimated time needed: **60** minutes

0.1.1 Overview:

In this lab, we will build a tool using Hidden Markov Model (HMM) to detect metamorphic malware. The tool will analyze opcode sequences and classify them as either malware or legitimate software.

0.1.2 Learning Objectives:

1. Learn to preprocess opcode sequences for machine learning.
2. Train a Hidden Markov Model using `hmmlearn`.
3. Handle errors and edge cases in model training.
4. Classify new opcode sequences using the trained model.

0.1.3 Implementation:

0.1.4 Step 1. Installing Required Libraries

Install the necessary library `hmmlearn` for Hidden Markov Models.

[1]:
```
#!pip install hmmlearn
```

0.1.5 step 2. Importing Required Libraries

Import libraries for data handling, preprocessing, and building the HMM model.

[2]:
```
#!pip install pandas
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from hmmlearn import hmm
import joblib
```

0.1.6 Step 3. Loading and Inspecting the Data

Load the opcode sequences from CSV files. Check the columns to ensure the structure is correct.

[3]:
```
# Load datasets
data1 = pd.read_csv('IDAN1.csv')
data2 = pd.read_csv('IDAN2.csv')
data3 = pd.read_csv('IDAN3.csv')

# Check columns in each dataset
# Write your code here!
print("Columns in data1:", data1.columns)
print("Columns in data2:", data2.columns)
print("Columns in data3:", data3.columns)
```

```
Columns in data1: Index(['Opcode', 'Operand', 'Comment'], dtype='object')
Columns in data2: Index(['Opcode', 'Operand', 'Comment'], dtype='object')
Columns in data3: Index(['Opcode', 'Operand', 'Comment'], dtype='object')
```

Click here for the solution

```
print("Columns in data1:", data1.columns)
print("Columns in data2:", data2.columns)
print("Columns in data3:", data3.columns)
```

0.1.7 Step 4. Data Preprocessing

Extract opcode sequences and encode them into numerical values using `LabelEncoder`.

[4]:
```
# Extract opcode sequences from each dataset
opcode_sequences1 = data1['Opcode'].astype(str)
opcode_sequences2 = data2['Opcode'].astype(str)
opcode_sequences3 = data3['Opcode'].astype(str)

# Convert each sequence into a list of opcodes
opcode_sequences1 = [sequence.split() for sequence in opcode_sequences1]
opcode_sequences2 = [sequence.split() for sequence in opcode_sequences2]
opcode_sequences3 = [sequence.split() for sequence in opcode_sequences3]

# Combine all opcode sequences into one list
combined_opcode_sequences = opcode_sequences1 + opcode_sequences2 +␣
 ↪opcode_sequences3

# Create a LabelEncoder to encode the opcodes
label_encoder = LabelEncoder()

# Flatten the list of lists to create a single list of all opcodes
all_opcodes = [opcode for sequence in combined_opcode_sequences for opcode in␣
 ↪sequence]
label_encoder.fit(all_opcodes)

# Encode each sequence using the label encoder
```

```
encoded_opcode_sequences = [label_encoder.transform(sequence) for sequence in␣
 ↪combined_opcode_sequences]

# Display a sample of the encoded sequences
print("Encoded Opcode Sequences Sample:", encoded_opcode_sequences[:3])
```

```
Encoded Opcode Sequences Sample: [array([236]), array([0]), array([8])]
```

Click here for the solution

```
# Combine all opcode sequences into one list
combined_opcode_sequences = opcode_sequences1 + opcode_sequences2 + opcode_sequences3

# Create a LabelEncoder to encode the opcodes
label_encoder = LabelEncoder()

# Flatten the list of lists to create a single list of all opcodes
all_opcodes = [opcode for sequence in combined_opcode_sequences for opcode in sequence]
label_encoder.fit(all_opcodes)

# Encode each sequence using the label encoder
encoded_opcode_sequences = [label_encoder.transform(sequence) for sequence in combined_opcode_s
```

Explanation: This code extracts opcode sequences from multiple datasets and converts each sequence into a list of individual opcodes. It then combines all sequences into one list and uses a LabelEncoder to convert the opcodes into numerical values for machine learning. Finally, it encodes each sequence into its numeric form, displaying a sample of the encoded sequences for verification.

0.1.8 Step 5. Concatenating Sequences and Defining Sequence Lengths

Concatenate the encoded sequences and define the lengths of each sequence.

[5]:
```
# Concatenate all encoded sequences into a single array
concatenated_sequences = np.concatenate(encoded_opcode_sequences)

# Store the lengths of each sequence
sequence_lengths = [len(sequence) for sequence in encoded_opcode_sequences]

# Display concatenated sequences and their lengths
# Write your code here!
# Display concatenated sequences and their lengths
print("Concatenated Sequences:", concatenated_sequences[:20])
print("Sequence Lengths:", sequence_lengths[:10])
```

```
Concatenated Sequences: [236   0   8   8 246 258   9 142 138 143 298 258 262   5
243   5 247 244
 237  12]
Sequence Lengths: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

Click here for the solution

```
# Display concatenated sequences and their lengths
print("Concatenated Sequences:", concatenated_sequences[:20])
print("Sequence Lengths:", sequence_lengths[:10])
```

Explanation: This code concatenates all encoded opcode sequences into a single NumPy array for easier manipulation during model training. It also calculates the length of each original sequence, storing these lengths in a list to help the Hidden Markov Model (HMM) understand the structure of the data. Finally, it prints a sample of the concatenated sequences and the lengths of the first ten sequences for verification and debugging purposes.

0.1.9 Step 6. Building the HMM Model

Define and initialize the HMM model. Ensure the transition matrix and start probabilities are set correctly.

[7]:
```
# Define number of hidden states for HMM (e.g., 2 for malware and legit)
n_components = 2

# Initialize HMM with specified parameters
# Write your code here!
model = hmm.MultinomialHMM(n_components=n_components, n_iter=100,␣
 ↪random_state=42)

# Set initial start probabilities and transition matrix
model.startprob_ = np.array([0.5, 0.5])  # Equal probability for both states␣
 ↪initially
model.transmat_ = np.array([
    [0.7, 0.3],  # Transition probabilities from state 0
    [0.3, 0.7]   # Transition probabilities from state 1
])
```

```
MultinomialHMM has undergone major changes. The previous version was
implementing a CategoricalHMM (a special case of MultinomialHMM). This new
implementation follows the standard definition for a Multinomial distribution
(e.g. as in https://en.wikipedia.org/wiki/Multinomial_distribution). See these
issues for details:
https://github.com/hmmlearn/hmmlearn/issues/335
https://github.com/hmmlearn/hmmlearn/issues/340
```

> **Note**: The warnings regarding the changes in MultinomialHMM reflect updates to the library's implementation. you can safely ignore these warnings, as the code will function correctly despite them.

Click here for the solution

```
# Initialize HMM with specified parameters
model = hmm.MultinomialHMM(n_components=n_components, n_iter=100, random_state=42)
```

Explanation: This code initializes a Hidden Markov Model (HMM) with two hidden states, which represent different classifications (e.g., malware and legit). The model is configured to iterate 100 times during training for optimal parameter fitting. Initial start probabilities are set to equal values, indicating an equal likelihood of starting in either state, while the transition matrix defines the probabilities of moving from one state to another, providing a foundational structure for the HMM to learn from the data.

0.1.10 Step 7. Training the HMM Model

Train the HMM on the concatenated sequences and sequence lengths. Check and fix the transition matrix if needed.

[8]:
```
# Train HMM model on encoded opcode sequences
model.fit(concatenated_sequences.reshape(-1, 1), sequence_lengths)

# Reinitialize transmat_ for rows that sum to zero
def reinitialize_transmat(transmat, epsilon=1e-5):
    for i in range(transmat.shape[0]):
        if transmat[i].sum() == 0:
            transmat[i] = np.full(transmat.shape[1], 1.0 / transmat.shape[1])
    return transmat

# Apply smoothing and reinitialize zero-sum rows in transmat_
# Write your code here!
# Apply smoothing and reinitialize zero-sum rows in transmat_
model.transmat_ = reinitialize_transmat(model.transmat_)
```

```
Even though the 'startprob_' attribute is set, it will be overwritten during
initialization because 'init_params' contains 's'
Even though the 'transmat_' attribute is set, it will be overwritten during
initialization because 'init_params' contains 't'
Some rows of transmat_ have zero sum because no transition from the state was
ever observed.
```

> **Note**: The warnings about startprob_ and transmat_ being overwritten are expected due to the initialization process in MultinomialHMM. Additionally, zero-sum rows in transmat_ indicate that no transitions were observed for those states, which is a common occurrence in HMMs and does not affect the model's functionality.

Click here for the solution

```
# Apply smoothing and reinitialize zero-sum rows in transmat_
model.transmat_ = reinitialize_transmat(model.transmat_)
```

Explanation: This code trains the Hidden Markov Model (HMM) using the encoded opcode sequences, reshaping the data to fit the model's input requirements and providing sequence lengths to guide the learning process. After training, a function is defined to reinitialize any rows in the transition matrix (`transmat_`) that sum to zero, ensuring that the model remains valid for future predictions. This function fills such rows with equal probabilities, preventing issues in the model's state transitions, and the reinitialized transition matrix is then updated in the model.

0.1.11 Step 8. Validating the Model

Validate the model's `startprob_` and `transmat_` to ensure they are correctly set.

[9]:
```
# Check and reinitialize startprob_ if it contains NaN
if np.isnan(model.startprob_).any():
    model.startprob_ = np.full(n_components, 1.0 / n_components)

# Verify startprob_ sums to 1
if not np.isclose(model.startprob_.sum(), 1.0):
    raise ValueError(f"Error: startprob_ must sum to 1 (got {model.startprob_.
 ↪sum()})")

# Check if the transition matrix is valid
def check_transmat(model):
    try:
        model._check()
        print("Transition matrix is valid.")
    except ValueError as e:
        print(f"Error: {e}")

# After training the model
# Write your code here!

check_transmat(model)
```

```
Transition matrix is valid.
```

Click here for the solution

```
# After training the model
check_transmat(model)
```

Explanation: This code snippet ensures the integrity of the model's initial state probabilities (`startprob_`) by checking for any NaN values. If found, it reinitializes them to equal probabilities, ensuring a valid starting point for the HMM. It then verifies that the sum of the initial probabilities equals 1, raising an error if it does not. Lastly, a function is defined to check the validity of the transition matrix (`transmat_`) after model training, confirming that the model parameters are correctly set up for subsequent predictions.

0.1.12 Step 9. Classifying New Opcode Sequences

Define a function to classify new sequences as malware or legitimate based on the trained model.

[10]:
```
def classify_opcode_sequence(opcode_sequence, trained_model, label_encoder):
    try:
        # Encode the sequence using the same label encoder
        encoded_sequence = label_encoder.transform(opcode_sequence)

        # Reshape to match the model input
```

```
        reshaped_sequence = np.array(encoded_sequence).reshape(-1, 1)

        # Compute the log likelihood for this sequence
        log_likelihood = trained_model.score(reshaped_sequence)

        # Based on log likelihood, classify as malware or legit
        if log_likelihood < -50:  # Adjust threshold based on your model's␣
↪performance
            return "Malware"
        else:
            return "Legit"
    except Exception as e:
        return f"Error: {str(e)}"

# Example usage
# Write your code here!

new_opcode_sequence = ["mov", "add", "jmp", "push"]  # Example sequence
prediction = classify_opcode_sequence(new_opcode_sequence, model, label_encoder)
print(f"\nPrediction for new sequence: {prediction}")
```

```
Prediction for new sequence: Legit
```

Click here for the solution

```
# Example usage
new_opcode_sequence = ["mov", "add", "jmp", "push"]  # Example sequence
prediction = classify_opcode_sequence(new_opcode_sequence, model, label_encoder)
print(f"\nPrediction for new sequence: {prediction}")
```

Explanation: This code defines a function to classify a given sequence of opcodes (instructions) as either "Malware" or "Legit" based on a trained Hidden Markov Model (HMM). It first encodes the input opcode sequence using the same `LabelEncoder` that was used during training, then reshapes the encoded data to fit the model's expected input format. The function calculates the log likelihood of the sequence using the trained model, which measures how likely the sequence is under the learned model parameters. If the log likelihood is below a specified threshold (in this case, -50), the sequence is classified as "Malware"; otherwise, it is classified as "Legit." The example usage demonstrates this classification process on a sample opcode sequence, printing the result.

In this instance, if the output is "Legit," it indicates that the model determined the provided opcode sequence does not resemble those typical of metamorphic malware, suggesting it is from a legitimate application.

0.1.13 Summary:

In this lab, you learned to build and train an HMM for detecting metamorphic malware based on opcode sequences. You implemented preprocessing, model training, and classification with a focus on handling common issues like invalid transition matrices.